

Mini Project IV

Section 1

Using the *wine dataset*,

1. **a)** A linear model with *Quality* as the response and using all predictors (with a factored Region) gave a Mean Squared Error (MSE):

simple linear model LOOCV MSE (bias_adjusted): 1.135158

- b) c) d) g)** Figure 1 below shows results from best subset selection:

```
call: regsubsets.formula(Quality ~ Aroma + Body + Flavor + Oakiness +
  Clarity + as.factor(Region), wineData, nvmax = 7)
7 variables (and intercept)
```

```
Forced in Forced out
Aroma FALSE FALSE
Body FALSE FALSE
Flavor FALSE FALSE
Oakiness FALSE FALSE
Clarity FALSE FALSE
as.factor(Region)2 FALSE FALSE
as.factor(Region)3 FALSE FALSE
```

```
1 subsets of each size up to 7
selection Algorithm: exhaustive
```

		Aroma	Body	Flavor	Oakiness	Clarity	as.factor(Region)2	as.factor(Region)3
1	(1)	" "	" "	"*	" "	" "	" "	" "
2	(1)	" "	" "	"*	" "	" "	"*	" "
3	(1)	" "	" "	"*	" "	" "	"*	"*
4	(1)	" "	" "	"*	"*	" "	"*	"*
5	(1)	"*	" "	"*	"*	" "	"*	"*
6	(1)	"*	"*	"*	"*	" "	"*	"*
7	(1)	"*	"*	"*	"*	"*	"*	"*

Figure 1

The best (maximum) adjusted R^2 model gave the model at index 4 from Figure 1, which is:

$$\text{WineQuality} = \beta_0 + \beta_1 \cdot \text{Flavor} + \beta_2 \cdot \text{Oakiness} + \beta_3 \cdot \text{as.factor(Region2)} + \beta_4 \cdot \text{as.factor(Region3)} + \epsilon$$

Figure 2

Surprisingly, the best forward selection and backward selection models were found to be of the exact same model as best subset selection.

They were all chosen by the best adjusted R^2 and I believe the 3 selection methods came to the same conclusion because there are only 6 predictors; R will use 2 dummy variables for the factored 3 Regions, so the true number of predictors I used was 7, but that still isn't many – especially for a dataset of 38 rows. The results are shown in Figure 3 below. It shows the equal adjusted R^2 values along with the same coefficients for each.

Method <chr>	Intercept <dbl>	Flavor <dbl>	Oakiness <dbl>	Region2 <dbl>	Region3 <dbl>	Adjusted_R2 <dbl>	Test_MSE <dbl>
Best Subset Selection	8.120817	1.192039	-0.3183165	-1.515484	1.093548	0.8164362	1.1094
Forward Stepwise Selection	8.120817	1.192039	-0.3183165	-1.515484	1.093548	0.8164362	1.1094
Backward Stepwise Selection	8.120817	1.192039	-0.3183165	-1.515484	1.093548	0.8164362	1.1094

3 rows

Figure 3

The test MSE was found using Leave-One-Out-Cross-Validation(LOOCV) which resulted in a test MSE of 1.1094. Since the selections all chose the same model, the test MSEs using LOOCV are all identical. Therefore, I can only recommend the model in Figure 2 for prediction based on the 3 selection methods as it had a lower MSE than the first simple linear model using all predictors.

e) Using Ridge Regression, the penalty parameter was chosen optimally using LOOCV and found to be 0.3162278; and the test MSE using that penalty parameter was 0.7004688.

f) Using Lasso, with the penalty parameter also chosen optimally using LOOCV, the penalty parameter was found to be 0.1293366; and the test MSE using that penalty parameter was 0.7154189.

Overall, I would recommend Ridge Regression or Lasso here with a preference for Lasso. Although Ridge Regression resulted in the lowest test MSE, Lasso offers a nearly equivalent test MSE with less than half of the penalty. Thus, Lasso balances flexibility and good predictive performance better than the rest for this dataset.

2.

a) Rewrite to:

$$\min_{\beta} [0.5(Y - X\beta)^T (Y - X\beta) + (\lambda / 2) \beta^T \beta]$$

Differentiate with respect to β and set equal to zero:

$$-X^T(Y - X\beta) + \lambda\beta = 0$$

Rearrange:

$$X^T Y = X^T X \beta + \lambda \beta$$

Factor:

$$(X^T X + \lambda I) \beta = X^T Y$$

With I being the Identity matrix:

$$\hat{B} = (X^T X + \lambda I)^{-1} X^T Y$$

b) Assuming errors are i.i.d.

$$Y_i = f(X_i) + \epsilon_i$$

For $i \neq j$:

$$\text{Cov}(Y_i, Y_j) = \text{Cov}(f(X_i) + \epsilon_i, f(X_j) + \epsilon_j)$$

Since errors are independent:

$$\text{Cov}(Y_i, Y_j) = \text{Cov}(f(X_i), f(X_j)) + \text{Cov}(\epsilon_i, \epsilon_j)$$

Since ϵ_i, ϵ_j are independent, $\text{Cov}(\epsilon_i, \epsilon_j) = 0$:

$$\text{Cov}(Y_i, Y_j) = \text{Cov}(f(X_i), f(X_j))$$

c)

$$Y^{\text{hat}}_i = X_i^T \beta^{\text{hat}}$$

Substituting β^{hat} from **a)**

$$Y^{\text{hat}}_i = X_i^T (X^T X + \lambda I)^{-1} X^T Y$$

d)

The fitted value Y^{hat}_i is a linear combination of actual values Y_j :

$$Y^{\text{hat}}_i = \sum_{j=1}^n w_{ij} Y_j$$

Therefore:

$$\text{Cov}(Y^{\text{hat}}_i, Y_i) = \text{Cov}(\sum_{j=1}^n w_{ij} Y_j, Y_i) = w_{ii} \text{Cov}(Y_i, Y_i)$$

Since $\text{Cov}(Y_j, Y_i) = \text{Var}(Y_i)$ and $\text{Cov}(Y_j, Y_i) = \text{Cov}(f(X_j), f(X_i))$ for $j \neq i$ then:

$$\sum_{j=1}^n \text{Cov}(Y^{\text{hat}}_i, Y_i) = \sum_{j=1}^n [\sum_{j=1}^n w_{ij} \text{Cov}(Y_j, Y_i)]$$

Section 2 R code

```
title: "Mini Project 4"
author: "Benny Frisella"
date: "2024-11-03"
output: pdf_document
```

```
` `` {r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
` ``
```

##1.

```
` `` {r}
wineData <- read.csv("wineData.txt", header = TRUE, sep = '\t')
` ``
```

(a) Fit a linear regression model using all predictors and compute its test MSE.

```
` `` {r}
```

```
library(boot)
```

```
#####1. (a)
```

```
#use LOOCV for all test MSE #
```

```
#take Quality as response with other 6 as predictors
```

```
wine_lm <- lm(Quality ~ Aroma + Body + Flavor + Oakiness +
as.factor(Region) + Clarity, data = wineData)
```

```
#perform LOOCV to compute test MSE using k folds where k = n
glm.fit <- glm(wine_lm, data = wineData)
```

```
cv.err <- cv.glm(wineData, glm.fit)
```

```
cat("Simple linear model LOOCV MSE (bias_adjusted):",  
cv.err$delta[1], '\n')
```

```
#Check higher polynomial models for better MSE#  
#try for Flavor  
cv.err_poly <- sapply(1:5, function(degree)  
{  
  fit <- glm(Quality ~ poly(Flavor, degree) + Body + Aroma  
    + Oakiness + as.factor(Region) + Clarity, data = wineData)  
  
  #LOOCV, extract bias_adjusted MSE  
  cv.glm(wineData, fit)$delta[1]  
})
```

```
#cv.err_poly  
#Note: A cubic model featuring Flavor^3 yields the lowest MSE:  
1.084790, this degree may be best fit for the data.
```

...

(b) Use best subset selection based on adjusted R-squared to find the best linear regression model. Compute the test MSE of the best model.

```
` `` {r}  
library(leaps)
```

#####1. (b)

```
#fit full model, 7 predictors because R adds 2 dummy variables for  
as.factor(Region)  
fit.full <- regsubsets(Quality ~ Aroma + Body  
  + Flavor + Oakiness + Clarity
```

```

+ as.factor(Region), wineData, nvmax = 7)
print(fit.full)

#gives the best model for each size
fit.summary <- summary(fit.full)

#get index of the model with the highest adjusted R-squared
best.model.index <- which.max(fit.summary$adjr2)

#display the summary of fits
print(fit.summary)

#plot adjusted R-squared
plot(fit.summary$adjr2,
     xlab = "Number of Variables",
     ylab = "Adjusted R-squared",
     type = "l")

points(best.model.index, fit.summary$adjr2[best.model.index],
       col = "red", cex = 2, pch = 20)

#gave model 4
` ``

` `` {r}
#try model with 2 predictors + 2 as.factor() predictors for region 2,3:
#subset the data to include only Region levels 2 and 3
wine_region_subset <- subset(wineData, Region %in% c(2, 3))

#fit the model using the subset data
wine_lm2 <- lm(Quality ~ Flavor + Oakiness + as.factor(Region), data =
wine_region_subset)

#fit generalized linear model

```

```

glm_fit2 <- glm(wine_lm2)

#use LOOCV to compute test MSE
cv_err2 <- cv.glm(wine_region_subset, glm_fit2)

#LOOCV MSE
cat("Model MSE (bias_adjusted) using Region levels 2 and 3,
    Flavor and Oakiness by best subset selection:", cv_err2$delta[1], '\n')

` ` `

Use forward stepwise selection based on adjusted R-squared
to find the best linear regression model.
Compute the test MSE of the best model.
` ` `{r}
#####1. (c)
fit.fwd <- regsubsets(Quality ~ Clarity + Body + Aroma + Flavor +
Oakiness
                    + as.factor(Region),
                    data = wineData, nvmax = 7, method = "forward")

#Best forward stepwise selection#

#gives the best model for each size
fit.summary2 <- summary(fit.fwd)

print(fit.summary2)

#get index of the model with the highest adjusted R-squared
best.model.index2 <- which.max(fit.summary2$adjr2)

#plot adjusted R-squared
plot(fit.summary2$adjr2,
     xlab = "Number of Variables",
     ylab = "Adjusted R-squared",

```

```

    type = "l")

points(best.model.index2, fit.summary2$adjr2[best.model.index2],
       col = "blue", cex = 2, pch = 20)

` ` `

` ` `{r}
#####1. (d)
fit.bwd <- regsubsets(Quality ~ Clarity + Body + Aroma + Flavor +
Oakiness
                     + as.factor(Region),
                     data = wineData, nvmax = 7, method = "backward")

# Best backward stepwise selection #

#gives the best model for each size
fit.summary3 <- summary(fit.bwd)

print(fit.summary3)

#get index of the model with the highest adjusted R-squared
best.model.index3 <- which.max(fit.summary3$adjr2)

#plot adjusted R-squared
plot(fit.summary3$adjr2,
     xlab = "Number of Variables",
     ylab = "Adjusted R-squared",
     type = "l")

points(best.model.index2, fit.summary3$adjr2[best.model.index3],
       col = "green", cex = 2, pch = 20)

` ` `

```


(e) Use ridge regression with penalty parameter chosen optimally via LOOCV to fit a linear regression model. Compute the test MSE of the model.

```
` `` {r}
```

```
#####1. (e)
```

```
library(glmnet)
```

```
# Prep #
```

```
x <- model.matrix(Quality ~ Clarity + Body + Aroma + Flavor + Oakiness  
+ as.factor(Region), data = wineData)[,-1]
```

```
y <- wineData$Quality
```

```
#compute MSE
```

```
mse <- function(actual, predicted)
```

```
{  
  mean((actual - predicted) ^ 2)  
}
```

```
#LOOCV to find the optimal lambda
```

```
#set.seed(8)
```

```
n <- nrow(wineData)
```

```
lambda_seq <- 10^seq(3, -3, by = -0.1) #sequence of lambda values
```

```
mse_values <- numeric(length(lambda_seq)) #store MSE for each  
lambda
```

```
for (i in 1:n)
```

```
{  
  train_indices <- -i  
  test_indices <- i
```

```
  x_train <- x[train_indices, ]
```

```
  y_train <- y[train_indices]
```

```

x_test <- x[test_indices, , drop = FALSE]
y_test <- y[test_indices]

ridge_model <- glmnet(x_train, y_train, alpha = 0, lambda =
lambda_seq)

#predict using the fitted model
preds <- predict(ridge_model, s = lambda_seq, newx = x_test)

#calculate MSE for each lambda
mse_values <- mse_values + colMeans((preds - y_test) ^ 2)
}

#average MSE across all n from LOOCV
mse_values <- mse_values / n

#optimal lambda
optimal_lambda <- lambda_seq[which.min(mse_values)]
cat("Optimal Lambda:", optimal_lambda, "\n")

#fit the final ridge regression model with the optimal lambda
final_ridge_model <- glmnet(x, y, alpha = 0, lambda = optimal_lambda)

#predict on the full data to calculate final MSE
final_preds <- predict(final_ridge_model, s = optimal_lambda, newx =
x)
final_test_mse <- mse(y, final_preds)

cat("Test MSE of the ridge regression model:", final_test_mse, "\n")

...

```{r}

```

```
#set up a function to compute the MSE
mse <- function(lambda)
{
 model <- glmnet(x, y, alpha = 1, lambda = lambda) #LASSO is alpha =
 1
 cv_error <- cv.glmnet(X, y, alpha = 1) #cross-validation
 return(cv_error$cvm[which.min(cv_error$lambda)])
}
```

```
#perform LOOCV to find the optimal lambda
#set.seed(8)
lasso_model <- glmnet(x, y, alpha = 1) #fit LASSO model
cv_fit <- cv.glmnet(x, y, alpha = 1) #cross-validation
optimal_lambda <- cv_fit$lambda.min #optimal lambda
```

```
#fit final model using the optimal lambda
final_lasso_model <- glmnet(x, y, alpha = 1, lambda = optimal_lambda)
```

```
#predict using the final model and compute MSE
predictions <- predict(final_lasso_model, newx = x)
mse_lasso <- mean((y - predictions)^2) #MSE
```

```
#output the results
cat("Optimal Lambda:", optimal_lambda, "\n")
cat("Test MSE of the LASSO regression model:", mse_lasso, "\n")
` ``
```

(g) Make a tabular summary of the parameter estimates and test MSEs from (a) - (c). Compare the results. Which model(s) would you recommend?

```
` `` {r}
Best models of size 4
```

testMSE = 1.1094

```

#create tabular structure
selection_table <- data.frame(
 Method = c("Best Subset Selection", "Forward Stepwise Selection",
 "Backward Stepwise Selection"),

 Intercept = c(coef(fit.full, 4)["(Intercept)"],
 coef(fit.fwd, 4)["(Intercept)"], coef(fit.bwd, 4)["(Intercept)"]),

 Flavor = c(coef(fit.full, 4)["Flavor"],
 coef(fit.fwd, 4)["Flavor"], coef(fit.bwd, 4)["Flavor"]),

 Oakiness = c(coef(fit.full, 4)["Oakiness"],
 coef(fit.fwd, 4)["Oakiness"], coef(fit.bwd, 4)["Oakiness"]),

 Region2 = c(coef(fit.full, 4)["as.factor(Region)2"],
 coef(fit.fwd, 4)["as.factor(Region)2"],
 coef(fit.bwd, 4)["as.factor(Region)2"]),

 Region3 = c(coef(fit.full, 4)["as.factor(Region)3"],
 coef(fit.fwd, 4)["as.factor(Region)3"],
 coef(fit.bwd, 4)["as.factor(Region)3"]),

 Adjusted_R2 = c(
 max(best_subset_summary$adjr2),
 max(forward_summary$adjr2),
 max(backward_summary$adjr2)),

 Test_MSE = c(testMSE, testMSE, testMSE)
)

print(selection_table)
` ``

```